



University
Mohammed VI
Polytechnic



Deliverable : Logical design Lab2

Data Management Course

UM6P College of Computing

Professor: Karima Echihabi **Program:** Computer Engineering

Session: Fall 2026

Team Information

Team Name	Team N°1
Member 1	Nizar El HAZZITI
Member 2	Mohamed JBARI
Member 3	Riyad HABIBI
Member 4	Chaimae IAA
Repository Link	https://github.com/Niz-arch/Data-Base-Project

1 Introduction

This lab focuses on the logical design phase of the RLMS project, which consists of translating the ER diagram built in Lab 1 into actual database tables with rows, columns and primary keys while preserving the semantics captured in the conceptual model.

2 Requirements

This deliverable translates the RLMS ER diagram into a relational schema, mapping all entity sets, relationships (including weak entities, ISA hierarchies, recursive and ternary relationships, and aggregation) into properly keyed tables, specifying primary keys, foreign keys. It also identifies any constraints not expressible via keys alone, and delivers the full .sql schema plus a short report on the team, challenges, and design choices.

3 Team roles

Nizar: Modelizing relational table for entities Person and Laboratories.

Chaimae: Modeling relational table for entities Calibration Record and Consumable

Riyad: Modelized relational table for the parts regarding certifications, reservations and maintenance.

4 Key challenges

-For Key constraints we won't need an individual table for such a relationship, therefore we add one attribute to the entity, otherwise we create another table for the relationship.

-For M:1 relationships (like monitors) no need to create a new table, we just add an attribute technicalId to the laboStock relationship, since each stock has only one monitor.

- "Supplier" is a ternary relationship with attribute "unitPrice", the attribute then depends on the combination of the three entities (lab, supplier, consumable), so we can't split it into labSupp and consumSupp.

-When it comes to Key constraints, for the part about reservations, personID, project code, and approverID were mentioned as attributes of the relation (reservations). After thinking about what guarantees unicity (meaning a at most one approver, exactly one person project), it appeared that the reservation ID itself guarantees it, given that it's its sole identifier. A single reservation, identified with its ID, is thus made by exactly one person, for exactly one project, approved by at most 1 person.

-Participation constraint for reservations was managed through using a table for the relationship (reserves), which receives as attribute the tuple (resID, serialNo), with serialNo defined as NOT NULL, which guarantees its existence (meaning that of the equipment unit), while not limiting it to one and one only (using the tuple property). Thus a reservation can be made for many units, but also at least one.

-Maintenance is given the primary key (startTS, serialNo), given that it is a weak entity, thus depending only on its owner's identifier. The startTS being its partial key, both allow us to identify a given maintenance operation, as each operation may be identified with its starting point and on what it's being conducted (the equipment unit).

-The technical staff ID is mentioned as well, as a NOT NULL attribute of (maintenance) (given the participation constraint). Since we also have a key constraint, exactly one staff member can conduct a given maintenance operation. The primary key (startTS, serialNo) guarantees the unicity of the staff member per operation.

5 Discussion

Lessons learned:

- We don't split a ternary relationship into two tables ! It stays one table with the 3 Foreign Keys .
- When merging a relationship into another table , we NEVER change the primary key (like monitors).
- A relationship based on top of an aggregated relationship (like stocks) must reference that aggregate's one table NOT the original entities it was built from.
- Always have a look onto what kind of constraint you have: never confound participation vs key constraints, as the two of them are treated differently. One may require us to drop a table (key constraint), the other may work just fine if not better with an additional table (participation constraint).
- Not to try and complicate things when it comes to key participation constraints. Sometimes we need to add some elements as primary keys to guarantee unicity, yet i noticed sometimes unicity may be guaranteed by something else (ie: for reservations, the resID guarantees it, thus no need to add other foreign keys as part of some primary key for reservations).

6 Conclusion and thoughts for next lab

A simple .sql file where we implementated the relational model. The only thing to address better is the consistency of code style, because the each member has his style (example: the use of camel or snake case, tabulations ...), we should find a good style and all of us use it.